TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

A Project Report On

Hallucination-Reduced Legal Contract Question Answering via Natural Language Inference and Knowledge Graphs

Submitted By:

Nirajan Sah (PUL078BCT054)

Prabin Adhikari (PUL078BCT058)
Rohan Thapa (PUL078BCT066)

Submitted To:

Department of Electronics & Computer Engineering

May 8th, 2026

Page of Approval

TRIBHUVAN UNIVERSIY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled **"Hallucination-Free Legal Contract Question Answering via Natural language Inference and Knowledge Graphs"** submitted by **Nirajan Sah**, **Prabin Adhikari**, **Rohan Thapa** in partial fulfillment of the requirements for the Bachelor's degree in Electronics & Computer Engineering.

.....	.....
Supervisor	External examiner
Asst. Prof. Ganesh Gautam	**Full Name**
Designation	Assistant Professor
Department of Electronics and Computer Engineering,	Department of Electronics and Computer Engineering,
Pulchowk Campus, IOE, TU.	Pulchowk Campus, IOE, TU.

Date of approval:

Copyright

Acknowledgments

First and foremost, we would like to express our sincere gratitude towards Pulchowk Campus, Institute of Engineering, including the Head of the Department and Deputy Head of the Department for providing us with a favorable environment and continued support for this project. Special Regards go to our Project Supervisor, Asst. Prof. Ganesh Gautam for constantly guiding us and providing us with valuable feedback at each step of our project duration. We would like to express our gratitude towards the Project Man- agement Team, incorporating IC Chairs Asst. Prof. Bikal Adhikari and others for their efforts, constant guidance and helpful encouragement. We would like to thank the Department of Electronics and Computer Engineering for providing us the opportunity of collaborative undertaking which has helped us to implement our knowledge as the Major Project for Final Year and developing a project of our own, which will greatly enhance our knowledge and provide us a new experience of teamwork.

Nirajan Sah (078BCT054)

Prabin Adhikari (078BCT058)

Rohan Thapa (078BCT066)

Abstract

Legal contract analysis is a critical yet labor-intensive process in the legal and business domains where precise interpretation of complex contractual language is required. Existing approaches to automated legal question answering (QA) suffer from hallucination, lack of verifiability, and inability to provide traceable reasoning. This project provides a pipeline that combines retrieval-augmented generation with multi-signal verification to produce accurate, verifiable answers from legal contracts.This system integrates a FAISS-based semantic retrieval engine using Sentence Transformers for clause-level retrieval, a fine-tuned Qwen 2.5 7B language model trained with Supervised Fine-Tuning (SFT) and Group Relative Policy Optimization (GRPO) for structured legal reasoning with tool-calling capabilities and fine-tuned DeBERTa-v3 Natural Language Inference (NLI) model for verification. The pipeline concludes with confidence gating that flags low-confidence answers, preventing silent hallucination.

Keywords: Legal NLP, Question Answering, Knowledge Graph, Retrieval-Augmented Generation, Neuro-Symbolic AI, Hallucination Detection, Natural Language Inference, Reinforcement Learning

Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CUAD	Contract Understanding Atticus Dataset
DeBERTa	Decoding-Enhanced BERT with Disentangled Attention
DPO	Direct Preference Optimization
DPR	Dense Passage Retrieval
FAISS	Facebook AI Similarity Search
GELU	Gaussian Error Linear Unit
GPT	Generative Pre-trained Transformer
GPU	Graphics Processing Unit
GQA	Grouped Query Attention
GRPO	Group Relative Policy Optimization
JSON	JavaScript Object Notation
KG	Knowledge Graph
KL	Kullback–Leibler Divergence
LLM	Large Language Model
MFN	Most Favored Nation
MLP	Multi-Layer Perceptron
NFR	Notice of Formation Rights
NLI	Natural Language Inference
NLP	Natural Language Processing
PEFT	Parameter-Efficient Fine-Tuning
PPO	Proximal Policy Optimization
QA	Question Answering
QWEN	Tongyi Qianwen (Qwen) Language Model Family
RAG	Retrieval-Augmented Generation
RL	Reinforcement Learning
RLC	Retrieval-Legal-Checker (Proposed Pipeline)
SBERT	Sentence-BERT

SFT	Supervised Fine-Tuning
SNLI	Stanford Natural Language Inference
STS	Semantic Textual Similarity
TRL	Transformer Reinforcement Learning
TU	Tribhuvan University
UML	Unified Modeling Language
VRAM	Video Random Access Memory
XML	Extensible Markup Language
XAI	Explainable Artificial Intelligence

1. Introduction

1.1 Background

The rapid advancement of artificial intelligence has transformed how organisations approach complex, knowledge-intensive tasks. Early machine learning approaches — including decision trees, logistic regression, and rule-based systems — were inherently interpretable by design. A practitioner could inspect such models directly and trace the logic behind any given output; these are commonly referred to as white-box models. However, the emergence of deep learning has fundamentally shifted this landscape. Modern neural architectures achieve stronger performance across a wide range of tasks, yet operate in ways that are largely opaque to the end user, earning the reputation of black-box models.

This tension between performance and interpretability is particularly pronounced in Natural Language Processing (NLP). Deep learning approaches to NLP rely on dense vector representations, commonly referred to as embeddings, of linguistic units such as words, subwords, or sentences, which are transformed through complex, high-dimensional operations to produce a final output. While the predictive capabilities of such architectures have been remarkable, the intermediate reasoning that leads to a particular prediction remains largely invisible. This growing recognition that model transparency matters has given rise to the field of Explainable AI (XAI), which seeks to provide users with meaningful insight into how a model arrives at its results.

In parallel, the legal domain has emerged as one of the most demanding application areas for NLP. Legal documents such as contracts, statutes, and case law are characterised by precise, domain-specific language, complex conditional structures, and long-range dependencies between clauses.

Retrieval-Augmented Generation (RAG) has emerged as a prominent paradigm for grounding large language model outputs in external document collections. Rather than relying solely on knowledge encoded in model weights during training, RAG systems retrieve relevant passages at inference time and condition generation on the retrieved context. This approach is particularly well-suited to legal contract QA, where answers must be directly traceable to specific clauses rather than inferred from general legal knowledge. Knowledge graphs offer a complementary representation, encoding structured relational facts extracted from text in a form amenable to symbolic reasoning and consistency checking. The combination of neural retrieval, large language model generation, and symbolic verification over a knowledge graph

represents the architectural foundation on which this work is built.

1.2 Problem statement

Legal contracts govern a significant portion of business relationships and transactions across organizations of all sizes. While larger enterprises employ legal professionals to analyze contractual obligations, rights, and restrictions, this remains relatively poorly accessible to smaller businesses due to cost. Even for trained professionals, the process of identifying and analyzing relevant clauses across lengthy contracts is labor-intensive and time-consuming.

This presents a clear opportunity for an automated NLP-based system capable of answering questions grounded in contract text. However, given that such a system's outputs would directly influence organizational decisions, raw model predictions alone are insufficient. A black-box system that returns an answer without justification offers little value to a non-technical stakeholder who needs to understand why a particular clause is relevant or how a conclusion was reached. The core challenge, therefore, is not only to build a system that answers legal contract queries accurately, but to do so in a way that is transparent and interpretable to a general audience and particularly top level management of any organization, whose technical background cannot be assumed to be strong. This requires the model to ground its answers in specific portions of the source contract, surfacing the exact clauses that support its output, rather than generating responses that cannot be traced back to the document. Achieving this in a domain as precise and consequential as legal text demands both retrieval accuracy and a principled approach to explainability.

1.3 Objectives

The objective of this project is to build an AI-based system that accepts a legal contract document and a natural language question as input, and produces an answer grounded in the source document. Critically, the system is designed not only to return a response, but to proved the specific clauses or elements that support it and provide verification signals that allow the user to assess the reliability of that response.

1.4 Scope

The system is scoped to English-language commercial contracts in plain text or PDF format. It processes one contract at a time; cross-contract querying is not supported. Clause extraction and knowledge graph construction are performed once at ingestion and are not updated incrementally as questions are asked. The knowledge graph uses a fixed set of Hohfeldian predicates defined at design time and does not infer predicates beyond those specified. The symbolic context retrieval used during query is keyword-based rather than semantic, and may miss relevant triples when question phrasing does not lexically overlap with extracted

entities.

The system extracts clauses, but only for it's look up purpose, it doesn't present it to the user, only mentions the clause index.

The system doesn't do the following:

- Does not provide legal advice, interpret enforceability, or apply statutory or case law
- Does not flag jurisdiction-specific issues or adapt interpretation based on governing law
- Does not detect internal contradictions between clauses within the same contract
- Does not flag missing or absent standard clauses
- Does not assess whether clauses are unusual, one-sided, or commercially risky
- Does not support multi-contract querying, comparison, or bulk processing
- Does not update the knowledge graph incrementally during a session
- Does not consult external legal databases, case law, or industry standards
- Does not adapt or retrain on user-uploaded contracts at runtime

2. Literature Review

2.1 Related Theory

2.1.1 Retrieval-Augmented Generation

Lewis et al. [1] introduced the RAG framework, which augments a language model's generation process by first retrieving relevant passages from an external corpus and conditioning the response on those passages. This separates knowledge storage from model parameters, allowing the system to answer questions grounded in a specific document rather than relying solely on what the model memorized during pretraining. This project adopts RAG as the core query architecture: a FAISS-indexed vector store of extracted clauses serves as the retrieval layer, and retrieved clauses are passed as context to the Qwen 2.5 7B answer generation model.

2.1.2 Dense Retrieval and Semantic Similarity

Karpukhin et al. [2] introduced Dense Passage Retrieval (DPR), demonstrating that dual-encoder architectures producing dense vector representations outperform sparse keyword-based methods for passage retrieval. Reimers and Gurevych [3] extended this with Sentence-BERT (SBERT), which uses siamese network fine-tuning to produce semantically meaningful sentence embeddings efficiently. This project uses the `all-MiniLM-L6-v2` distilled SBERT variant to embed contract clauses at ingestion time and embed user queries at inference time, enabling meaning-based clause retrieval rather than keyword matching. Johnson et al. [4] introduced FAISS, providing efficient approximate nearest neighbor search over these dense embeddings. This project uses a flat inner product FAISS index, providing exact cosine similarity search over the clause embedding store.

2.1.3 Knowledge Graphs and Hohfeldian Legal Predicates

A knowledge graph represents information as a set of structured triples: subject, predicate, object, capturing explicit named relationships between entities. Unlike dense embeddings, this structured representation supports relational queries and consistency checks that neural models cannot reliably perform alone. Hohfeld [5] formalized a vocabulary of fundamental legal relations: obligation, permission, prohibition, power, and immunity, that provides a principled basis for structuring legal relationships symbolically. This project uses Hohfeldian

predicates including `OBLIGATED_TO`, `PERMITTED_TO`, `PROHIBITED_FROM`, `HAS_POWER_TO`, `PAYS`, `TERMINATES`, `GOVERNS`, and `HAS_DEADLINE` as the predicate vocabulary for the per-contract knowledge graph, which is then used to cross-check structural claims in generated answers. Gutierrez et al. [6] demonstrated that LLMs can extract structured triples from text with high accuracy, which is the mechanism this project uses to populate the graph via the Claude API at ingestion time. Pan et al. [7] surveyed LLM-KG integration paradigms; this project falls into their synergized category, where the LLM and KG mutually inform each other during the pipeline.

2.1.4 Hallucination in Large Language Models

Ji et al. [8] categorized LLM hallucinations as intrinsic , where the output contradicts the source and extrinsic where it cannot be verified from the source. Blair-Stanek et al. [9] demonstrated that even GPT-4 makes errors in approximately 20% of cases involving complex conditional logic in legal texts, and Savelka et al. [10] showed persistent hallucination on legal tasks despite GPT-4's general capability. These findings motivate the verification layer in this project: rather than treating model output as reliable, every claim in a generated answer is passed through a separate verification stage before being returned to the user.

2.1.5 NLI-Based Claim Verification

Natural Language Inference (NLI) is the task of determining whether a premise text entails, contradicts, or is neutral toward a hypothesis. He et al. [11] introduced DeBERTa with disentangled attention, achieving state-of-the-art NLI performance. Honovich et al. [12] demonstrated that NLI models can be repurposed for factual consistency verification, given a source passage as premise and a generated claim as hypothesis, the model judges whether the passage actually supports the claim. Laban et al. [13] showed that fine-tuned NLI models outperform general-purpose models for domain-specific verification tasks. Thorne et al. [14] established the retrieve-then-verify pipeline as a standard fact-checking architecture. This project uses a fine-tuned DeBERTa-v3 NLI cross-encoder as the neural verification component: each atomic claim in the generated answer is checked against its cited clause before the answer is surfaced to the user.

2.1.6 Supervised Fine-Tuning and GRPO

Supervised fine-tuning (SFT) adapts a pretrained language model to a target output format by training on curated input-output pairs. This project uses SFT to train Qwen 2.5 7B [15] on a legal tool-calling format, teaching the model to structure its reasoning as a sequence of operations: cite, verify, extract, and summarize. Shao et al. [16] introduced Group Relative Policy Optimization (GRPO), a reinforcement learning method that generates K responses

per prompt, scores them, and updates the policy based on relative ranking within the group, removing the need for a separate critic model. This project applies GRPO after SFT using $K = 4$ rollouts and a five-signal hybrid reward: NLI grounding (30%), citation accuracy (20%), completeness (15%), format compliance (15%), and Claude-as-judge (20%).

2.1.7 Parameter-Efficient Fine-Tuning with LoRA

Hu et al. [17] introduced LoRA (Low-Rank Adaptation), which freezes pretrained model weights and injects trainable low-rank decomposition matrices into each transformer layer, reducing trainable parameters by orders of magnitude while maintaining fine-tuning quality. This project uses LoRA with rank 32 for both SFT and GRPO stages on Qwen 2.5 7B, reducing the trainable parameter count from 7.6 billion to approximately 83 million, making training feasible without full model fine-tuning infrastructure.

2.2 Related Work

2.2.1 Legal Contract Understanding

Chalkidis et al. [18] introduced LEGAL-BERT, a BERT model pre-trained on European Union legal text, achieving strong results on legal NLP benchmarks. LEGAL-BERT establishes that domain-specific pretraining improves performance on legal text tasks. This project does not use a domain-pretrained base model but instead relies on retrieval and post-generation verification to achieve grounding, using a general-purpose instruction-tuned model as the generator.

Hendrycks et al. [19] introduced CUAD, a dataset of 13,000+ expert annotations across 41 contract clause types from 510 commercial contracts. CUAD-based systems are designed for clause classification and span extraction on a fixed set of predefined categories. This project addresses a different task, open-ended question answering over an arbitrary uploaded contract, and does not assume a fixed clause taxonomy.

Bommarito and Katz [20] and Savelka et al. [10] evaluated GPT-3.5 and GPT-4 directly on legal tasks, finding useful capability but persistent hallucination on precision-critical questions. These works apply LLMs as direct answer generators without a retrieval or verification layer. This project treats raw LLM output as unverified by design and adds both a retrieval stage to ground generation and a dual verification stage to check outputs before delivery.

2.2.2 Legal RAG Systems

Wiratunga et al. [21] demonstrated that clause-level semantic retrieval significantly outperforms document-level retrieval for contract analysis, validating the clause-granularity inges-

tion design used in this project. Louis et al. [22] proposed a legal RAG system combining statutory text with case law retrieval. Both systems focus on retrieval quality as the primary mechanism for improving answer accuracy. This project extends beyond retrieval quality by adding a post-generation verification layer, retrieval is necessary but not treated as sufficient for correctness.

2.2.3 Legal Knowledge Graph Systems

Lam et al. [23] constructed a legal knowledge graph from Hong Kong court judgments, demonstrating that structured representations enable complex relational queries over legal text. Dragoni et al. [24] proposed ontology-driven triple extraction from contractual texts. Both works build knowledge graphs for querying legal relationships directly. This project uses a knowledge graph differently, not as the primary query interface, but as a structural cross-check against neural generation, verifying that entity relationships stated in a generated answer are consistent with the triples extracted from the contract.

2.2.4 Automated Verification Systems

Manakul et al. [25] proposed SelfCheckGPT, which detects hallucinations by sampling multiple responses and checking consistency across them. Min et al. [26] introduced FActScore, which decomposes generated text into atomic facts and verifies each independently against a source. Both systems approach verification as a purely neural task. This project combines neural verification (NLI cross-encoder) with symbolic verification (KG structural check), treating them as complementary: the NLI model catches textual contradictions while the KG check catches relational inconsistencies that surface only when the answer is compared against the structured graph. Sansford et al. [27] propose GraphEval, a hallucination evaluation framework that represents LLM responses as Knowledge Graphs (KGs) and applies NLI models at the level of individual graph triples. By checking each atomic triple against the source knowledge independently, GraphEval both improves balanced accuracy over raw NLI baselines on standard hallucination benchmarks and provides fine-grained localisation of which specific claims in a response are inconsistent. This triple-level decomposition aligns closely with the atomic claim verification strategy adopted in this project, and demonstrates that pairing structured claim extraction with NLI verification yields more reliable and explainable hallucination detection than applying NLI to whole-response text.

3. Methodology

3.1 Requirement Analysis

3.1.1 Functional Requirements

Functional requirements define what a system should do.It outlies specific functionalities and feaures that the system must exhibit to mee the user needs.the funcitonal requirements are:

1. **FR-01 (Contract Ingestion):** The system shall accept contract text and segment it into individual clauses using Claude API, preserving exact original text.

2. **FR-02 (Triple Extraction):** The system shall extract subject-predicate-object triples from each clause using Hohfeldian legal predicates and store them in a knowledge graph.

3. **FR-03 (Clause Retrieval):** The system shall encode clauses into dense vectors and retrieve the top-K most relevant clauses for a given question using FAISS cosine similarity.

4. **FR-04 (Symbolic Context):** The system shall query the knowledge graph for triples relevant to the user question and assemble them as structured context for the LLM.

5. **FR-05 (Answer Generation):** The system shall generate structured answers using the fine-tuned Qwen 2.5 7B model with tool-call format (cite $\rightarrow$ verify $\rightarrow$ extract $\rightarrow$ summarize).

6. **FR-06 (NLI Verification):** The system shall verify each extracted claim against retrieved clause text using the fine-tuned DeBERTa-v3 NLI model, outputting entailment, contradiction, and neutral probabilities.

7. **FR-07 (KG Cross-Check):** The system shall verify claims against KG triples by checking entity and predicate overlap, providing a structural consistency signal.

8. **FR-08 (Confidence Gating):** The system shall compute an overall confidence score from weighted verification results and attach a warning label if confidence falls below 0.5.

9. **FR-09 (Web Interface):** The system shall provide a Streamlit-based web interface where users can upload contracts, ask questions, and view verified answers with confidence scores.

3.1.2 Non-Functional Requirements

The requirements which should be there in system in order to enhance the working of the system can be referred as non-functional requirements. some of the non functional requirements of the system can be stated as:

1. **NFR-01 (Response Time):** The system shall generate and verify an answer within 60 seconds on a GPU-equipped machine (excluding initial model loading).

2. **NFR-02 (Accuracy):** The NLI verification model shall achieve $\geq$60% 3-class accuracy and $\geq$0.70 Spearman correlation on the evaluation set.

3. **NFR-03 (Hallucination Detection):** The system shall correctly flag $\geq$70% of hallucinated claims with contradiction scores above 0.5.

4. **NFR-04 (Scalability):** The FAISS index shall support contracts with up to 500 clauses without degradation in retrieval latency.

5. **NFR-05 (Transparency):** Every claim in the output shall be traceable to a source clause ID and accompanied by a verification score and method.

6. **NFR-06 (Memory Efficiency):** The system shall operate within 24 GB GPU VRAM during inference by using 4-bit quantization and LoRA adapters.

7. **NFR-07 (Reproducibility):** All training configurations, hyperparameters, and random seeds shall be documented and logged via Weights & Biases.

8. **NFR-08 (Extensibility):** The modular pipeline design shall allow individual components (retriever, generator, verifier) to be replaced or upgraded independently.

3.2 Process model

For the development of this project, we used the prototyping model.Prototyping model is one of the most popularly used Software Development Life Cycle Models where a prototype of the end product is first developed,tested and refined as per the feedback repeatedly till a final acceptable prototype is achieved.

This model was choosen because of the experimental and iterative nature of machine learning projects.

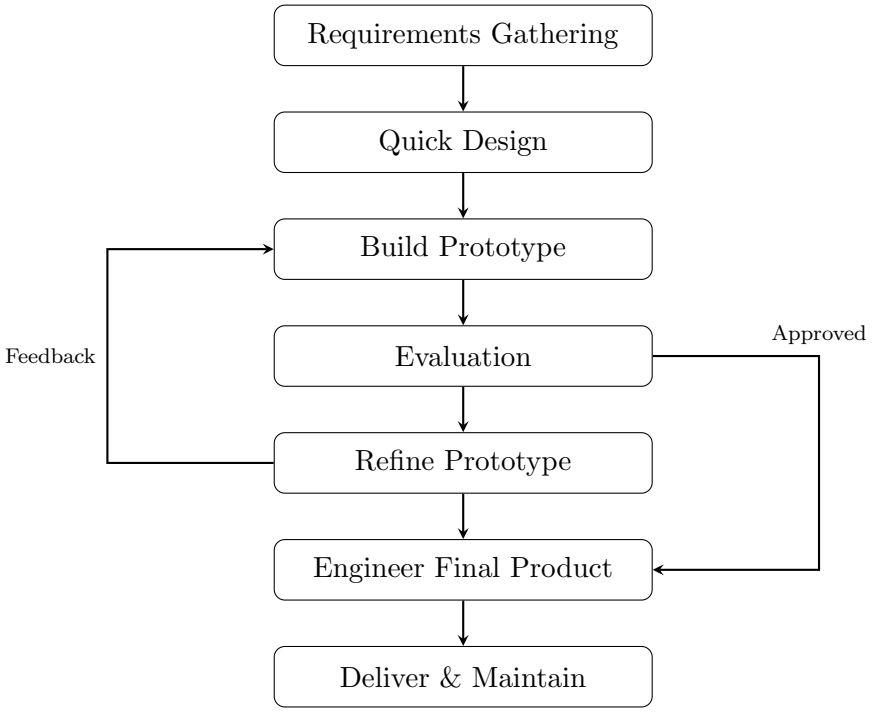


Figure 3.1: Prototyping Model

3.2.1 Development Phases

The development followed prototyping iterations:

Prototype 1: Mamba and Deep-Problog Mamba (state-space model) was used for efficient long-document processing and answer generation, while DeepProbLog provided neuro-symbolic verification by encoding legal rules as probabilistic logic programs with neural predicates.

Identified issues: Mamba lacked instruction-following capabilities, DeepProbLog required manual rule-crafting per contract and couldn't handle ambiguous legal language, and integrating both systems was brittle.

Prototype 2: Structured Generation Fine-tuned Qwen 2.5 7B with SFT on tool-calling format. Implemented structured output parsing (cite → verify → extract → summarize).

Identified issues: incorrect citations, no post-generation verification.

Prototype 3: NLI Verification Trained DeBERTa-v2 NLI model on legal entailment pairs. Added NLI-based claim verification.

Identified issues: misses structural errors (wrong amounts, dates).

Prototype 4: Knowledge Graph Integration Implemented KG construction with triple

extraction. Added KG cross-check as second verification signal.

Identified issues: limited KG coverage for some predicate types.

Prototype 5: GRPO and Full Integration Applied GRPO training with 5-signal hybrid reward. Integrated all components into end-to-end pipeline. Added confidence gating and Streamlit application. Conducted final evaluation and ablation studies.

3.3 Sentence Transformers (all-MiniLM-L6-v2

Sentence Transformers [3] is a framework for computing dense vector representations of sentences.These produce fixed-size sentence embeddings rather than the token-level embeddings by standard BERT which suitable for semantic similarity computation.

The **all-MiniLM-L6-v2 model** is a distilled version of the MiniLM architecture. It consists of 6 transformer layers with a hidden dimension of 384 and 12 attention heads, totalling approximately 22 million parameters.This model accept up to 256 tokens and uses mean pooling over all token embeddings to produce a single vector represenation.

Table 3.1: all-MiniLM-L6-v2 Specifications

Parameter	**Value**
Architecture	MiniLM (distilled from BERT)
Layers	6 transformer layers
Hidden Dimension	384
Attention Heads	12
Parameters	∼22 million
Max Sequence Length	256 tokens
Pooling Strategy	Mean pooling
Similarity Metric	Cosine similarity
STS Benchmark Score	0.8420

The embedding process completes in following stages:

- **Tokenization stage:** The input text is tokenized using a WordPiece tokenizer
- **Token Embedding stage:** Tokens pass through the 6-layer transformer encoder
- **Pooling stage:** mean pooling is applied to all the token embeddings to produce a single 384-dimensional vector
- **Normalization stage:** The output vector from the previous sstage is then L2-normalized so that inner product equals cosine similarity.

This model was choosed becauses of its strong quality-to-efficient ratio. It achieves competitive performance on semantic similarity and benchmarks while being fast enough for real-time retrieval.

3.4 FAISS -Facebook AI Similarity Search

FAISS [4] is a library for efficient similarity search and clustering of dense vectors.The IndexFlatIP (Flat Inner Product) index was used in this project, which performs exact neares neighbor search. As the vectors are L2-normalized (from the sentence transformer), the inner product equals cosine similarity:

$$\cos(\theta) = \frac{q \cdot c}{\|q\| \cdot \|c\|} = q \cdot c \quad (\text{when } \|q\| = \|c\| = 1) \tag{3.1}$$

Table 3.2: FAISS Index Configuration

Parameter	**Value**
Index Type	IndexFlatIP (Flat Inner Product)
Dimension	384
Search Type	Exact (brute-force)
Top-K	5 (configurable)
Threshold	0.3 (minimum similarity)

All clause embeddings are added to the FAISS index during ingestion. During the query time , the question is encoded, and FAISS computes the inner product between the query vector and all clause vector. The top-K clauses with similarity score above thatn threshold are returned.

3.5 QWEN 2.5 7B Instruct

Qwen 2.5 7B Instruct [15] is a 7-billion parameter instruction-tuned language model from the Qwen family developed by Alibaba Cloud. This model is used as the base model for the answer generation of the project.

Table 3.3: Qwen 2.5 7B Instruct Architecture

Parameter	**Value**
Architecture	Transformer Decoder (GPT-style)
Parameters	7.62 billion
Layers	32 transformer layers
Hidden Dimension	4,096
Attention Heads	32
Key-Value Heads	8 (Grouped Query Attention)
Intermediate Size	11,008
Vocabulary Size	151,936
Max Context Length	131,072 tokens
Activation	SwiGLU
Position Encoding	RoPE (Rotary Position Embedding)
Normalization	RMSNorm

This model uses several architectural innovations. Grouped Query Attention (GQA) users 8 key-value heads which are shared across 32 attention heads. This reduce memory usage while maintaning quality. SwiGLU activation is used rather than standard ReLU or GELU because of the better gradient flow.RoPE(Rotary Position Embedding) enables effective handling of long sequences through relative position encoding. The instruct variant has been fined-tuned on intruction-following data so this makes it suitable for our project for domain specific fine-tuning.

This model demonstrated competitive reasoning capabilities at 7B parameters. It fits within a single GPU with bfloat16 precision and supports a 128K token context window for lengthy contract clause. It is also compatible with LoRA fine-tuning. Because of these feasibility , this model was chosen. For training, the 4-bit quantized variant (unsloth/qwen2.5-7b-instruct-unsloth-bnb-4bit) was used via the Unsloth framework to fit withina single NVIDIA L4 GPU(24 GB RAM).

3.6 LoRA Fine-Tuning

LoRA [17] is a parameter-efficient fine-tuning technique that freezes the pre-trained model weights and injects trainable rank-decomposition matrices.For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA adds:

$$W = W_0 + \Delta W = W_0 + BA \tag{3.2}$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$, with rank $r \ll \min(d, k)$. Only B and A are trained while W_0 remains frozen.

Table 3.4: LoRA Configuration

Parameter	**Value**
Rank (r)	32
Alpha (α)	64
Scaling factor (α/r)	2.0
Target Modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Trainable Parameters	80,740,352 (1.62% of total)
Total Parameters	4,972,287,488 (frozen) + 80.7M (trainable)
Quantization	4-bit (via Unsloth + BitsAndBytes)
Dropout	0.05

There are several advantages of LoRA in this project: only around 1.62% of parameters are trainable which reduced the GPU memory significantly.Training is faster, the adapter is modular and with rank 32 targeting all attention and MLP projections, it captures sufficient task-specific information for legal reasoning

3.7 Supervised Fine-Tuning

SFT is the first phase of the two-phase training pipeline. The Qwen 2.5 7B model (4- bit quantized via Unsloth) is trained on 1,053 valid legal QA pairs (filtered from 1,274 original examples by removing impossible-to-fit sequences) where each example contains contract clauses, a question, and an expected response in a structured tool-calling format. The model learns to generate responses using four tools in a specific chain: (1) `cite_clause` to quote exact text from the contract with section references, (2) `verify_claim` to verify a factual claim against evidence, (3) `extract_fact` to extract a key fact with a confidence score, and (4) `summarize_analysis` to produce a final answer with key findings. Each tool call follows an XML format with named arguments, and the expected chain is always: cite first, then verify, then extract, then summarize.

The training data consists of legal contract QA pairs from the CUAD-based reasoning data and synthetic generation, where each example contains contract clauses, a question and the expected structured tool-calling response. The model is fine-tned using the standard causal language modeling with the LoRA adapter described in previous section.

3.8 Group Relative Policy Optimization (GRPO)

GRPO is the second training phase, applied after SFT to further align the model with desired behaviors. Unlike PPO which requires a separate critic model, GRPO generates multiple responses per prompt and uses their relative reward rankings to update the policy.

3.8.1 GRPO Algorithm

For each training prompt x, the algorithm: (1) samples $K = 4$ responses from the current policy, (2) scores each response using the hybrid reward function, (3) normalizes advantages within the group as $\hat{A}_i = \frac{R(y_i) - \text{mean}(R)}{\text{std}(R)}$, and (4) updates the policy using the clipped surrogate objective:

$$\mathcal{L}(\theta) = -\mathbb{E}\left[\min\left(\frac{\pi_\theta(y|x)}{\pi_{\theta_{\text{old}}}(y|x)}\hat{A},\ \text{clip}\left(\frac{\pi_\theta(y|x)}{\pi_{\theta_{\text{old}}}(y|x)}, 1-\epsilon, 1+\epsilon\right)\hat{A}\right)\right] \tag{3.3}$$

3.8.2 5-Signal Hybrid Reward Function

The total reward combines five complementary signals:

$$R(y|x) = 0.30 \cdot R_{\text{NLI}} + 0.20 \cdot R_{\text{cite}} + 0.15 \cdot R_{\text{complete}} + 0.15 \cdot R_{\text{format}} + 0.20 \cdot R_{\text{judge}} \tag{3.4}$$

Table 3.5: GRPO 5-Signal Hybrid Reward Function

Signal	**Weight**	**Description**
NLI Grounding	30%	Cross-encoder entailment score of claims vs. contract text
Citation Accuracy	20%	Proportion of cited text appearing verbatim in the contract
Completeness	15%	Whether the tool chain includes all four required steps
Format	15%	Whether tool calls follow the expected XML format
Claude Judge	20%	LLM-based quality, coherence, and accuracy assessment

The GRPO training is built on the SFT adapter,selecting high-quality ,category-balanced examples from the training set. The training uses individual response backward passes as a VRAM optimization where backward pass for each of the responses are computed separately to keep the only one computation graph in memory at a time.

3.9 DeBERTa-v3 for NLI

The NLI verification model is based on DeBERTa-v3-base (`microsoft/deberta-v3-base`, He et al., 2021), which improves upon BERT through **disentangled attention** — separate content and position embeddings that interact through three attention terms:

$$A_{ij} = H_i^c (H_j^c)^T + H_i^c (H_j^p)^T + H_i^p (H_j^c)^T \tag{3.5}$$

This decomposition better captures the interaction between content meaning and positional context.

Table 3.6: DeBERTa-v3-base NLI Model Specifications

Parameter	**Value**
Architecture	DeBERTa-v3-base (disentangled attention)
Base Model	`microsoft/deberta-v3-base`
Hidden Dimension	768
Intermediate Size	3,072
Layers	12 transformer layers
Attention Heads	12
Vocabulary Size	128,100
Parameters	$\sim$86 million
Classification Head	Linear ($768 \rightarrow 3$)
Output Classes	Entailment (1), Contradiction (0), Neutral (2)
Max Sequence Length	512 tokens
Attention Dropout	0.1
Hidden Dropout	0.1

At inference time, the model takes a premise (contract clause text, up to 512 tokens) and a hypothesis (a claim about the contract) and outputs probability scores for all three classes. The grounding score (entailment probability) is the primary metric used by the verification pipeline.

3.10 Knowledge Graph Construction and Triple Extraction

The Knowledge Graph is a structured representation of the contract's key facts, stored as subject-predicate-object triples with Hohfeldian legal predicates.

Table 3.7: Knowledge Graph Triple Predicate Schema

Category	**Predicates**	**Example**
Deontic	OBLIGATED_TO, PERMITTED_TO, PROHIBITED_FROM	(Provider)-[OBLIGATED_TO]→(deliver services)
Potestative	HAS_POWER_TO, TERMINATES	(Either Party)-[HAS_POWER_TO]→(terminate with 60 days notice)
Transactional	PAYS, DELIVERS	(Client)-[PAYS]→($50,000 monthly)
Assignment	ASSIGNS, RETAINS, GRANTS	(Provider)-[ASSIGNS]→(all IP rights to Client)
Structural	GOVERNS, LIMITS_LIABILITY_TO	(Agreement)-[GOVERNS]→(laws of New York)
Conditional	CONDITIONED_ON, HAS_DEADLINE	(Payment)-[HAS_DEADLINE]→(30 days from invoice)

For each clause in the contract, an LLM extracts structured triples using a prompt that specifies the predicate schema. Each triple maintains a reference to its source clause ID, enabling full traceability. The extracted triples are stored in an in-memory graph and visualized as a directed graph with color-coded edges indicating predicate categories. From the demo contract (a 9-section services agreement), the system typically extracts 20–40 triples covering all major contractual relationships.

3.11 Multi-Signal Verification Framework

The core innovation of this project is the multi-signal verification framework, which combines two independent verification signals.

Table 3.8: Multi-Signal Verification Decision Matrix

Method	**Condition**	**Weight**	**Verified?**
NLI+KG (strongest)	NLI > 0.5 AND KG confirmed	1.0	Yes
NLI only	NLI > 0.7	0.9	Yes
KG only	KG confirmed, NLI < 0.7	0.8	Yes
WEAK	NLI 0.4–0.7, no KG	0.6	No
UNVERIFIED	NLI < 0.4, no KG	0.4	No

The overall confidence is computed as:

$$\text{Confidence} = \frac{\sum_{i=1}^{N} w_i \cdot \text{NLI}_i}{\sum_{i=1}^{N} w_i} \tag{3.6}$$

Confidence Gating: If the overall confidence is below 0.5, the answer is delivered with an explicit **WARNING** label indicating low confidence and specifying how many claims could not be verified. This prevents silent hallucination by ensuring uncertain answers are always flagged.

3.12 Dataset Preparation

3.12.1 CUAD v1: Base Dataset

The dataset used is CUAD v1 (Contract Understanding Atticus Dataset). This dataset is an expert-annotated legal benchmark with 510 commercial contracts,41 clause categories, and roughly 13k+ clause annotations.Each contract is annotated with extractive QA pairs where annotatins highlight relevant text spand for 41 distinct legal clause types.

Table 3.9: CUAD v1 Dataset Statistics

Metric	**Count**
Commercial Contracts	510
Clause Categories	41
Total QA Pairs	20,910
Total Clause Annotations	13,823
Answerable Questions	6,702
Unanswerable (Impossible) Questions	14,208
Average Answers per Answ. Question	2.06

3.12.2 Clause categories

CUAD v1 covers 41 distinct legal clause categories spanning contract metadata, intellectual property, liability, termination and regulatory compliance. The categories can be shown as below:

Table 3.10: CUAD Clause Categories (1–28)

#	**Category Name**	**Description**
1	Document Name	Name of the contract document
2	Parties	Entities signing the contract
3	Agreement Date	Date contract was agreed upon
4	Effective Date	Date contract becomes effective
5	Expiration Date	Date contract expires
6	Renewal Term	Terms for contract renewal
7	Termination For Convenience	Either party may terminate without cause
8	Notice Period To Terminate Renewal	Required notice before termination
9	Affiliate License-Licensor	Licensor subsidiary rights
10	Affiliate License-Licensee	Licensee subsidiary rights
11	License Grant	Permission to use IP/assets
12	Non-Transferable License	License cannot be transferred
13	Irrevocable Or Perpetual License	Permanent or non-revocable license
14	Unlimited/All-You-Can-Eat-License	Unlimited usage rights
15	Ip Ownership Assignment	Transfer of IP ownership
16	Joint Ip Ownership	Shared IP ownership between parties
17	Source Code Escrow	Code held in escrow arrangement
18	Minimum Commitment	Minimum performance/purchase required
19	Volume Restriction	Limits on usage/purchase volume
20	Exclusivity	Exclusive rights granted
21	Most Favored Nation	MFN clause equality terms
22	Cap On Liability	Liability limit amount
23	Uncapped Liability	Unlimited liability exposure
24	Liquidated Damages	Pre-agreed damages for breach
25	Price Restrictions	Pricing constraints or caps
26	Revenue/Profit Sharing	Revenue/profit distribution terms
27	Governing Law	Jurisdiction and applicable law
28	Insurance	Insurance requirements

Table 3.11: CUAD Clause Categories (29–41)

#	Category Name	Description
29	Non-Compete	Restrict competition during/after contract
30	No-Solicit Of Customers	No solicitation of counterparty clients
31	No-Solicit Of Employees	No solicitation of counterparty staff
32	Warranty Duration	How long warranties are valid
33	Anti-Assignment	Restrictions on assigning contract rights
34	Change Of Control	Rights upon ownership change
35	Audit Rights	Right to audit compliance
36	Post-Termination Services	Services provided after termination
37	Rofr/Rofo/Rofn	Right of first refusal/offer/negotiation
38	Covenant Not To Sue	Agreement not to initiate legal action
39	Third Party Beneficiary	Non-party benefits from contract
40	Competitive Restriction Exception	Exceptions to competition limits
41	Non-Disparagement	Cannot make negative public statements

3.12.3 Dataset transformation

For dataset construction from CUAD to match our objective, we primarily used Anthropic Claude to produce and refine high-quality training examples. First, the CUAD annotations are converted into NLI-style premise-hypothesis pairs, capturing entailment, contradiction, and neutral relationships.After that the dataset is reformatted into tool-calling prompts that include ground-truth evidence spans along with expected reasoning traces.Additionally the impossible or unanswerable labelled cases are incorporated to discourage hallucinated outputs. Finally, during supervised fine-tuning , only samples with valid reasoning traces and answerable contexts are retained to enforce a structured generation format, while GRPO with the help of hybrid reward based on grounding, citation accuracy, completeness, and output format consistency, it selects a high-quality subset and optimizes responses.

Training Data Examples

The following examples show actual training data :

SFT Training Data Format Each SFT training sample follows a chat-format with system, user, and assistant messages. The assistant response contains structured tool calls in the cite → verify → extract → summarize sequence.

```
{
  "messages": [
```

```
    {
      "role": "system",
      "content": "You are a legal reasoning model..."
    },
    {
      "role": "user",
      "content": "**Contract:**\n[contract text]\n\n
                  **Question:** Can the agreement be assigned
                  without consent?"
    },
    {
      "role": "assistant",
      "content": "<tool_call>
        {\"name\": \"cite_clause\",
         \"arguments\": {\"clause_id\": \"cl_6.3\",
           \"text\": \"Neither party shall assign...\"}}
      </tool_call>
      <tool_call>
        {\"name\": \"verify_claim\",
         \"arguments\": {\"claim\": \"Assignment is prohibited
           without written consent\",
           \"evidence\": \"shall not assign...\"}}
      </tool_call>
      <tool_call>
        {\"name\": \"extract_facts\",
         \"arguments\": {\"facts\": [
           {\"name\": \"assignment_prohibited\",
            \"value\": 0.95}]}}
      </tool_call>
      <tool_call>
        {\"name\": \"summarize\",
         \"arguments\": {\"answer\": \"No, the agreement
           cannot be assigned without prior written
           consent of the other party...\"}}
      </tool_call>"
    }
  ]
}
```

Listing 3.1: SFT Training Sample Format (simplified)

NLI Training Data Format The NLI training data consists of premise-hypothesis pairs with entailment labels, derived from legal contract clauses.

```
// Positive example (entailment)
{
  "premise": "Either Party may terminate this Agreement
    upon sixty (60) days prior written notice.",
  "hypothesis": "Either party can terminate with
    60 days notice.",
  "label": 1,
  "score": 1.0
}
```

```
10
11 // Hard negative (contradiction)
12 {
13   "premise": "Either Party may terminate this Agreement
14     upon sixty (60) days prior written notice.",
15   "hypothesis": "Only the client can terminate
16     the agreement.",
17   "label": 0,
18   "score": 0.0
19 }
20
21 // Nuanced (neutral)
22 {
23   "premise": "The Provider shall deliver software
24     development services as described in Exhibit A.",
25   "hypothesis": "The Provider must deliver all
26     services within 30 days.",
27   "label": 2,
28   "score": 0.5
29 }
```

Listing 3.2: NLI Training Data Samples

GRPO Reward Signal Format Each GRPO training step generates $K = 4$ responses per prompt, scores them with the 5-signal reward function, and computes group-normalized advantages.

```
1  Prompt: "What is the payment structure?"
2
3  Response 1: reward = 0.82
4    NLI: 0.91, Citation: 0.85, Complete: 1.0,
5    Format: 1.0, Judge: 0.65
6
7  Response 2: reward = 0.71
8    NLI: 0.78, Citation: 0.70, Complete: 1.0,
9    Format: 0.75, Judge: 0.55
10
11 Response 3: reward = 0.45
12   NLI: 0.42, Citation: 0.35, Complete: 0.50,
13   Format: 1.0, Judge: 0.40
14
15 Response 4: reward = 0.63
16   NLI: 0.65, Citation: 0.60, Complete: 1.0,
17   Format: 0.50, Judge: 0.55
18
19 Group statistics:
20   mean = 0.6525, std = 0.1368
21   Advantages: [+1.22, +0.42, -1.48, -0.16]
22   -> Response 1 reinforced most strongly
23   -> Response 3 penalized most strongly
```

Listing 3.3: GRPO Reward Computation Example

4. Experimental Setup

4.1 Training Environment

Table 4.1: Hardware and Software Environment

Category	**Tool/Library**	**Purpose**
Runtime	Lightning AI Studio	Cloud compute environment
GPU	NVIDIA L4 (24 GB VRAM)	Training and inference
Language	Python 3.12	Primary development language
Deep Learning	PyTorch 2.1.0+	Neural network framework
Transformers	HuggingFace Transformers 4.40+	Model loading and inference
Fine-Tuning	PEFT (LoRA)	Adapter management
RL Training	TRL	GRPO training
Embeddings	Sentence Transformers	Clause embedding
Vector Search	FAISS (faiss-gpu)	Similarity search index
Graph	NetworkX	In-memory knowledge graph
Visualization	Matplotlib	Charts and diagrams
Web App	Streamlit	Interactive demo application
Quantization	BitsAndBytes	Mixed precision inference
Acceleration	Accelerate	Multi-GPU and bfloat16

4.2 Training Datasets

Table 4.2: Training Data Summary

Dataset	Size	Description
SFT Training Data	1,053 valid pairs (from 1,274)	Legal contract QA pairs wi tool-calling traces. Filtered quence length (3,072 token tokens/example.
GRPO Training Data	300 samples	High-quality, category-bal set. Each prompt generate sponses scored by the 5-sig function.
NLI Training Data	2,190 train / 161 eval	Contract claim-evidence pa as entailment, contradictio tral. From 8,655 gold + original pool.
CUAD, grounding, and synthetic datasets.		

4.3 Training Configuration

4.3.1 SFT Training Hyperparameters

The base model used in the project -`unsloth/qwen2.5-7b-instruct-unsloth-bnb-4bit`- is loaded in 4-bit quantization via the Unsloth framework, reducing VRAM requirements to fit on a single NVIDIA L4 GPU. The LoRA adapter adds 80.7M trainable parameters (1.62% of total 4.97B parameters).

Table 4.3: SFT Training Configuration

Parameter	**Value**
Dataset Size	1,053 valid pairs (from 1,274 original)
Evaluation Set	182 valid pairs (from 225 original)
Average Tokens per Example	997 (train), 988 (eval)
Max Total Tokens	1,607 (train), 1,461 (eval)
Learning Rate	2×10^{-4}
Epochs	2
Per-Device Batch Size	1
Gradient Accumulation Steps	8 (effective batch size = 8)
Max Sequence Length	3,072 tokens
Max Prompt Length	2,048 tokens
Max Completion Length	768 tokens
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.95$)
Weight Decay	0.01
Warmup Ratio	5% (13 warmup steps)
Scheduler	Linear warmup + Cosine decay
Max Gradient Norm	1.0
Gradient Checkpointing	Yes (Unsloth optimized)
Quantization	4-bit (BitsAndBytes)
Loss Function	Cross-entropy (next-token prediction)

4.3.2 GRPO Training Hyperparameters

GRPO training starts from the SFT adapter and further aligns the model using reinforcement learning with the 5-signal hybrid reward.

Table 4.4: GRPO Training Configuration

Parameter	**Value**
Training Samples	300 (high-quality selection, category-balanced)
Evaluation Samples	225
Starting Point	SFT adapter (loaded on base model)
Responses per Prompt (K)	4
Learning Rate	5×10^{-6}
Per-Device Batch Size	1
Gradient Accumulation Steps	4
Clip Range (ϵ)	0.2
KL Coefficient	0.04
Temperature (sampling)	0.8
Top-p	0.9
Top-k	50
Max New Tokens	384
Epochs	1
Scheduler	OneCycleLR (cosine annealing)
Warmup Ratio	5%
Weight Decay	0.01
Max Gradient Norm	1.0
Claude Judge Model	claude-haiku-4-5 (sampled at 25% rate)
Quantization	4-bit (BitsAndBytes via Unsloth)

4.3.3 NLI Model Training Hyperparameters

The DeBERTa-v3-base model (`microsoft/deberta-v3-base`, 86M parameters) is fine-tuned as a 3-way classifier on legal contract claim-evidence pairs.

Table 4.5: NLI Model Training Configuration and Results

Parameter	**Value**
Training Data	2,190 pairs (from 8,655 gold + weak label pool)
Evaluation Data	161 pairs (from 962 original)
Positive Examples (eval)	61
Hard Negative Examples (eval)	58
Nuanced Examples (eval)	42
Epochs	10
Final Results (Epoch 10)	
3-Class Accuracy	64.6%
F1-Macro	0.632
F1-Weighted	0.653
Binary Accuracy	78.9%
Binary F1	0.750
Spearman Correlation	0.722
Pearson Correlation	0.678
Positive Accuracy	70.5% (mean entailment prob: 0.663)
Hard Negative Accuracy	70.7% (mean entailment prob: 0.072)
Nuanced Accuracy	47.6% (mean entailment prob: 0.299)

Training loss curves and detailed per-epoch metrics are presented in Chapter 6 (Result and Analysis).

4.4 Evaluation Data

The system is evaluated on a held-out test set containing 225 legal QA examples across 41 categories, sourced from real-world commercial contracts (CUAD-based). Due to compute constraints, 10 examples were evaluated quantitatively.

Table 4.6: Evaluation Dataset Overview

Parameter	**Value**
Total evaluation examples	225
Total categories	41
Examples evaluated (due to compute)	10
Each example includes	Question, contract text, expected tool trace
Source contracts	Real-world commercial contracts (CUAD-based)

Table 4.7: Top-10 Evaluation Categories by Count

Category	**Count**
Exclusivity	11
Notice Period To Terminate Renewal	10
Audit Rights	10
Uncapped Liability	9
Competitive Restriction Exception	8
Minimum Commitment	8
Non-Transferable License	8
Termination For Convenience	8
Parties	8
Covenant Not To Sue	7

4.5 Evaluation Metrics

The following metrics are used to evaluate the system:

1. **Tool Trace F1**: F1 score comparing the set of generated tool names against the expected tool sequence from the ground truth. Measures whether the model follows the correct reasoning chain.

2. **Format Score**: Proportion of tool calls that follow the correct XML format with proper argument names. Measures structural correctness of the model's output.

3. **NLI Grounding Score**: Average entailment probability across all claims in a response, as scored by the fine-tuned DeBERTa-v3 model. Measures factual alignment with the source contract.

4. **Tool Usage Rates**: Percentage of responses that use each of the three primary tools (cite_clause, verify_claim, summarize_analysis).

5. **Confidence**: Weighted average of NLI scores using verification method weights. Reflects overall answer reliability.

6. **Grounding Rate**: Proportion of claims verified by NLI+KG, NLI-only, or KG-only (i.e., not WEAK or UNVERIFIED).

5. System design

5.1 High-Level System Architecture

The system architecture consists of two major phases:Ingestion (one-time per contract) and Queery (per question).

During the Ingestion Phase ,the raw contract text is first divided into distinct legal clauses which is carried out once for each contract. A knowledge graph is created by extracting structured subject-predicate-object triples from each clause. Sentence Transformers are used to simultaneously encode each clause into 384-dimensional dense vectors, which are then indexed in an FAISS vector store for quick retrieval.

During the Query Phase ,the user's query is encoded and used to extract the top-K most semantically similar clauses from the FAISS index which is carried out per question. Relevant triples from the knowledge graph are assembled as symbolic context. The refined Qwen 7B model receives the question along with the retrieved clauses and symbolic context. It then uses a four-tool reasoning chain to produce an organized response. Two independent tests are then used to verify each claim in the generated response: KG cross-checking (matching against structured triples) and NLI entailment checking (using the refined DeBERTa-v3 model). Lastly, low-confidence responses are flagged by the confidence gating mechanism, which calculates a weighted confidence score.

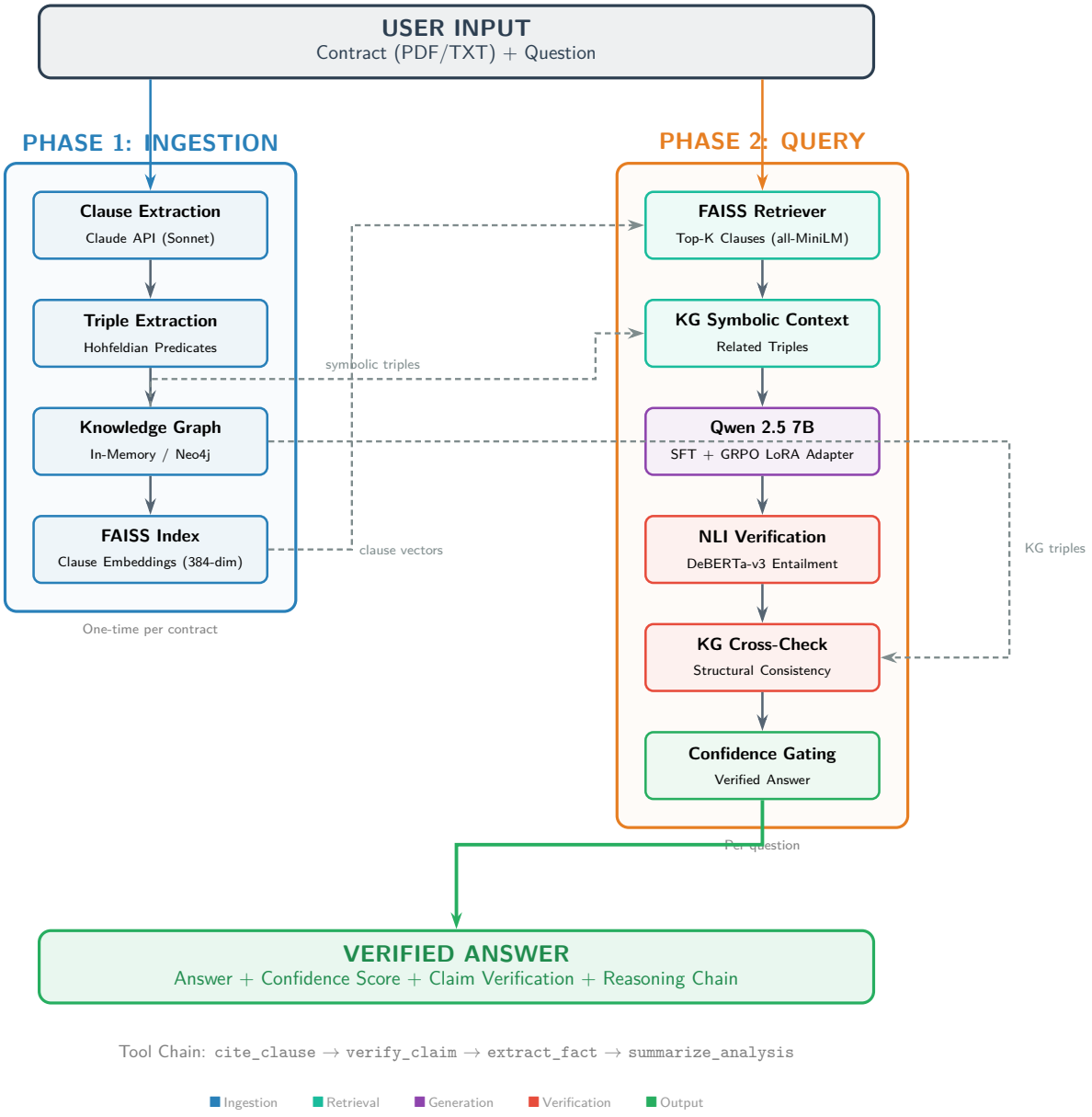


Figure 5.1: System Architecture — Ingestion and Query Phases

5.2 Data Flow Description

The data flows through the system as follows:

1. Contract text → Clause Extractor → Individual Clauses
2. Individual Clauses → Triple Extractor → Triples → Knowledge Graph
3. Individual Clauses → Sentence Transformer → Embedding Vectors → FAISS Index

4. User Question → Sentence Transformer → Query Vector
5. Query Vector + FAISS Index → Top-K Relevant Clauses
6. Question + Knowledge Graph → Symbolic Context (Related Triples)
7. Top-K Clauses + Symbolic Context + Question → Fine-tuned Qwen 7B → Structured Response with Tool Calls
8. Extracted Claims + Clause Text → DeBERTa-v3 NLI Model → Entailment Scores
9. Extracted Claims + KG Triples → KG Cross-Checker → Structural Consistency Results
10. NLI Scores + KG Results → Confidence Gate → Verified Answer with Confidence Score

5.3 Data Models

The system uses five core data models. **Clause** represents a single contract clause with a unique identifier (e.g., `demo_acme:cl_2.1`), title, text, and section number. **Triple** represents a KG fact with subject, predicate, object, source clause reference, and confidence score. **ToolStep** represents a parsed tool call from the model output with step number, tool name, and input/output dictionaries. **VerificationMethod** is an enumeration of the five verification tiers: NLI+KG, NLI, KG, WEAK, and UNVERIFIED. **VerifiedClaim** associates a claim string with its verification result, method, NLI score, KG status, and evidence.

These dataclasses are implemented using Python's `dataclasses` module and flow through the entire pipeline, providing a consistent data contract between components.

5.4 Activity Diagram

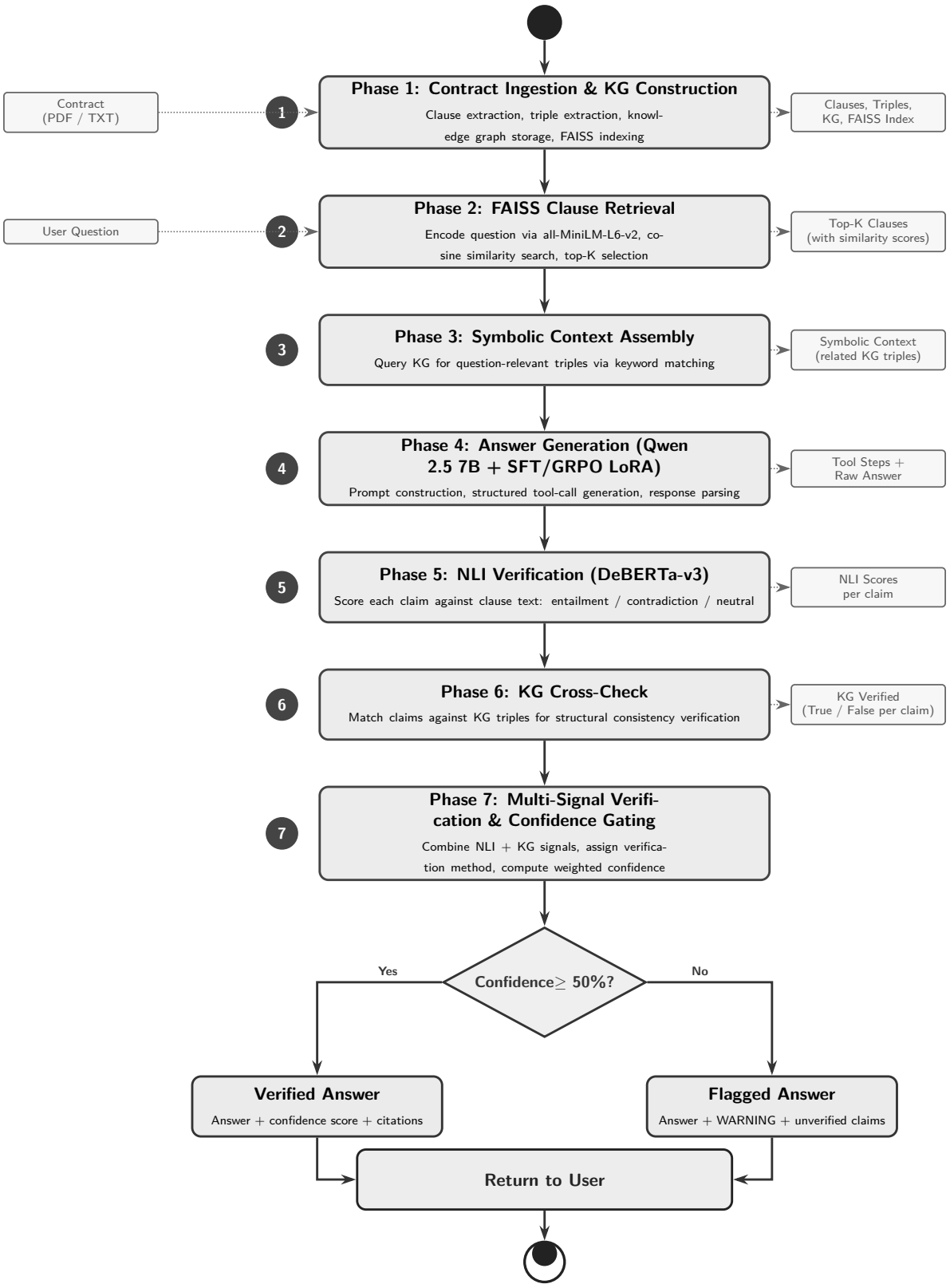


Figure 5.2: Activity Diagram

5.4.1 Phase 1: Contract Ingestion and Knowledge Graph Construction

The ingestion phase begins with **clause extraction**, which segments the raw contract text into individual, semantically coherent clauses. Each clause is assigned a unique identifier following the pattern `CONTRACT_ID:cl_SECTION_NUMBER` (e.g., `demo_acme_techserv:cl_2.1`). The extraction preserves the exact original text without any paraphrasing, ensuring source fidelity.

Next, **triple extraction** is performed on each clause. The extraction uses a structured prompt that specifies the Hohfeldian predicate schema and requests output as JSON containing subject, predicate, and object fields. Each extracted triple maintains a reference to its source clause ID, enabling full traceability.

Table 5.1: Example Extracted Triples from the Demo Contract

Subject	**Predicate**	**Object**	**Source**
Provider	OBLIGATED_TO	deliver software development and consulting services	cl_1
Agreement	HAS_DEADLINE	24 months Initial Term	cl_2.1
Either Party	HAS_POWER_TO	terminate with 60 days written notice	cl_2.2
Client	PAYS	Monthly Fee of $50,000	cl_3.1
Payment	HAS_DEADLINE	30 days from invoice	cl_3.2
Work Product	ASSIGNS	sole and exclusive property of Client	cl_4.1
Provider	RETAINS	pre-existing tools and frameworks	cl_4.3
Agreement	GOVERNS	laws of State of New York	cl_7

The knowledge graph is visualized as a directed graph where nodes represent entities and edges represent predicates, with color coding indicating predicate categories (red for obligations, green for permissions, blue for powers, yellow for transactional relations).

5.4.2 Phase 2: FAISS Clause Retrieval

During ingestion, all clause texts (prepended with their title if available) are encoded into 384-dimensional dense vectors using the Sentence Transformer model. The vectors are L2-normalized and added to a FAISS `IndexFlatIP` index.

At query time, the user's question is encoded using the same Sentence Transformer, and FAISS computes the inner product (cosine similarity) between the query vector and all clause vectors. The top-K results (K=5 by default) with scores above the threshold of 0.3 are returned as the retrieved context.

5.4.3 Phase 3: Symbolic Context Assembly

Before generating an answer, the system queries the knowledge graph for structured facts relevant to the user's question. This is done by keyword matching — significant words from the question (longer than 3 characters) are compared against the text of each triple's subject, predicate, and object fields. Matching triples are assembled into a structured context string that provides the LLM with verified facts from the KG, complementing the retrieved clause text.

For example, for the question "What is the termination notice period?", the symbolic context would include triples such as `(Either Party)-[HAS_POWER_TO]->(terminate with 60 days written notice)`, providing the model with a verified fact anchor.

5.4.4 Phase 4: Answer Generation with Fine-Tuned Qwen 7B

The fine-tuned model is loaded in two stages: first the base Qwen 2.5 7B Instruct model (in bfloat16 precision for GPU deployment), then the SFT LoRA adapter is applied on top. The model is set to evaluation mode with no gradient computation for inference efficiency.

The system prompt defines the model's role as a legal contract analysis agent and specifies the four available tools with their expected argument formats. The user message is constructed by concatenating the retrieved clause texts (with clause IDs and section numbers), the symbolic context from the KG, and the user's question.

The prompt is formatted using the Qwen chat template and the model generates with low temperature (0.1) and top-p sampling (0.9) to produce deterministic, high-quality responses of up to 512 new tokens. The raw response is then parsed to extract individual tool calls using regex pattern matching, producing a sequence of `ToolStep` objects representing the model's reasoning chain.

In the demo, the model consistently generated 4-step reasoning chains (cite $\rightarrow$ verify $\rightarrow$ extract $\rightarrow$ summarize) with generation times ranging from 24.1s to 30.8s on the GPU.

5.4.5 Phase 5: NLI Verification

The NLI verification module loads the fine-tuned DeBERTa-v3 model and scores each extracted claim against the concatenated text of the retrieved clauses (truncated to 512 tokens). For each premise-hypothesis pair, the model outputs a softmax probability distribution over three classes: entailment, contradiction, and neutral. The entailment probability serves as the **grounding score**.

5.4.6 Phase 6: KG Cross-Check

The KG cross-check provides a structural consistency signal independent of the NLI model. For each claim, the system checks whether both the subject and object of any KG triple appear in the claim text, or whether either entity appears along with predicate-related words.

The KG cross-check catches structural errors that NLI might miss — for example, if the model states "Provider pays $40,000/month" but the KG has `(Client)-[PAYS]-$50,000`, both the entity and amount mismatches would prevent confirmation. Conversely, claims that paraphrase correctly but involve entities not in the KG will not be found, which is why the dual NLI+KG approach is necessary.

5.4.7 Phase 7: Multi-Signal Verification and Confidence Gating

For each claim extracted from the model's output, the system computes both the NLI grounding score and the KG cross-check result. Based on the decision matrix (Table 3.8), each claim is assigned a verification method (NLI+KG, NLI, KG, WEAK, or UNVERIFIED). Claims verified by NLI+KG, NLI alone, or KG alone are marked as grounded; WEAK and UNVERIFIED claims are not.

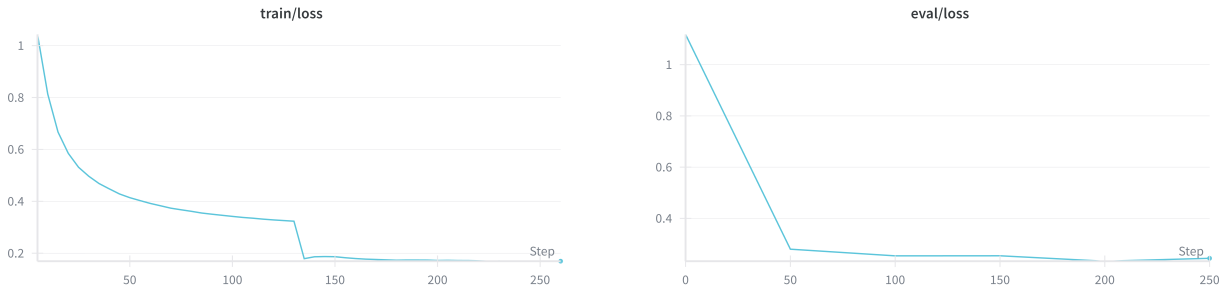
The overall confidence score is computed as a weighted average of NLI scores, where the weight for each claim depends on its verification method. If the confidence falls below 0.5, the answer includes an explicit warning indicating low confidence and specifying how many claims were not verified.

6. Results & Discussion

6.1 Results

6.1.1 SFT Training Results

The SFT trainingdone on NVIDIA L4 GPU was completed with zero out-of-memory errors. The training loss decreased from 1.118 to 0.169 at final while the evaluation loss was converges to a best of 0.2329 at step 200.



(a) SFT Training Loss

(b) SFT Evaluation Loss

Figure 6.1: SFT Training and Evaluation Loss Curves over 2 Epochs (263 steps). Training loss shows rapid convergence in Epoch 1 (1.118 → 0.323) with continued refinement in Epoch 2 (0.323 → 0.169). Evaluation loss reaches a minimum of 0.2329 at step 200.

Table 6.1: SFT Training Progression

Checkpoint	**Train Loss**	**Eval Loss**	**Status**
Pre-training	1.118	—	Baseline
Epoch 1 (avg)	0.323	—	—
Step 50 eval	—	0.280	New best
Step 100 eval	—	0.254	New best
Step 150 eval	—	0.254	—
Step 200 eval	—	**0.233**	**Best model saved**
Step 250 eval	—	0.244	—
Epoch 2 (avg)	0.169	—	Final

6.1.2 GRPO Training Results

GRPO training was ran for 1 epoch around 300 steps for about 2.5 hours on NVIDIA L4 GPU. The result can be shown in given table.

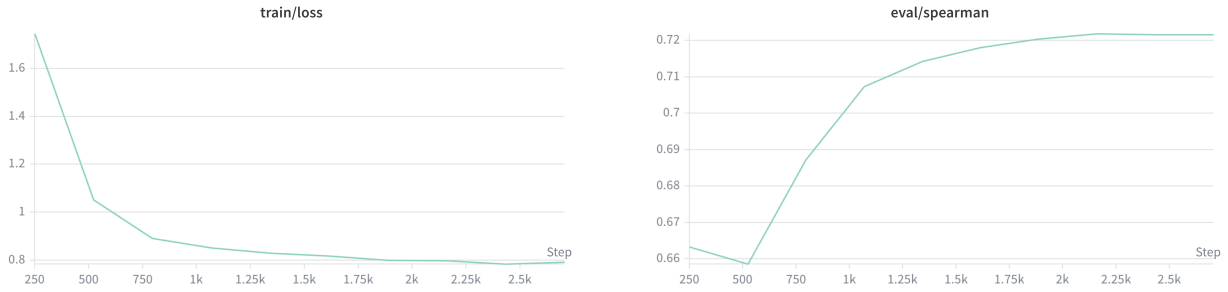
Table 6.2: GRPO Training Progression (Selected Steps)

Step	Loss	Reward	NLI	Claude	KL
0 (pre-train)	—	0.699	0.450	0.590	—
10	0.0007	0.718	0.377	0.600	0.0004
20	0.0001	0.845	0.816	0.575	−0.0007
30	0.0015	0.859	0.814	0.650	0.0011
40	0.0004	0.851	0.886	0.500	0.0003
50 (eval)	—	0.666	0.368	0.635	—
100 (eval)	—	0.612	0.293	0.535	—

The best evaluation reward was observed at step 50 with the value of 0.666. The KL divergence remained near zero throughout training, indicating the policy stayed close to the SFT baseline while improving on the reward signals.

6.1.3 NLI Model Training Results

The DeBERTa-v3-base NLI model was trained for 10 epochs on 2,190 legal contract claim-evidence pairs.



(a) NLI Training Loss (per epoch)

(b) NLI Evaluation Spearman Correlation

Figure 6.2: NLI Model Training Curves over 10 Epochs. Training loss decreases from 1.745 to 0.793 with rapid improvement in early epochs. Spearman correlation on the evaluation set improves from 0.663 to 0.722, plateauing after epoch 6.

Figure 6.3: NLI Training Step Loss (fine-grained). Shows the per-step loss decreasing from ∼2.0 to ∼0.8 over 2,700 training steps.

Table 6.3: NLI Model Training Results per Epoch (Selected Epochs)

Epoch	**Train Loss**	**Eval Loss**	**3-Class Acc**	**F1 Macro**	**Binary Acc**	**Spearman**
1	1.745	1.572	60.2%	0.591	77.6%	0.663
3	0.891	0.847	63.4%	0.621	77.6%	0.687
5	0.829	0.831	65.2%	0.637	78.9%	0.714
7	0.800	0.828	65.2%	0.637	78.9%	0.720
10	0.793	0.830	64.6%	0.632	78.9%	**0.722**

Table 6.4: NLI Model Final Performance by Example Type (Epoch 10)

Example Type	**Count**	**Accuracy**	**Mean Entailment Prob**
Positive (entailed)	61	70.5%	0.663
Hard Negative (contradicted)	58	70.7%	0.072
Nuanced (ambiguous)	42	47.6%	0.299
Overall	**161**	**64.6%**	**0.355**

Strong discrimination between hard negative and positive examples is demonstrated by the model (mean entailment probability 0.663 vs. 0.072). At 47.6% accuracy, new complex examples continue to be the most difficult category. This is to be expected given that they involve ambiguous legal language, making it difficult to determine the proper classification.

6.1.4 NLI Verification Results

Table 6.5: NLI Verification Demo — Grounded vs. Hallucinated Claims

Claim	**Type**	**Entail.**	**Contra.**	**Neutral**
Either party can terminate with 60 days notice	Grounded	0.817	0.035	0.148
Only the client can terminate the agreement	Hallucinated	0.008	0.921	0.070
The termination notice period is 90 days	Hallucinated	0.022	0.897	0.081
Breach must be cured within 10 days	Hallucinated	0.016	0.874	0.110

The NLI model successfully discriminates between grounded claims (high entailment score of 0.817) and hallucinated claims (high contradiction scores of 0.874–0.921), demonstrating its effectiveness for legal hallucination detection.

6.1.5 KG Cross-Check Results

Table 6.6: KG Cross-Check Demo Results

Claim	**Result**	**KG Evidence**
Client shall pay Provider a monthly fee of $50,000	CONFIRMED	(Client)-[PAYS]->(Monthly Fee of $50,000)
Provider is obligated to deliver software services	CONFIRMED	(Provider)-[OBLIGATED_TO]->(deliver services)
The agreement is governed by laws of Delaware	NOT FOUND	—
The liability cap is $1,000,000	NOT FOUND	—
Payment is due within 60 days	NOT FOUND	—

The KG cross-check correctly confirms claims that match extracted triples (e.g., $50,000 monthly fee) while rejecting claims with wrong entities ("Delaware" vs. actual "New York"), wrong amounts ("$1,000,000"), or wrong values ("60 days" vs. actual "30 days"). Claims

involving predicates or entities not extracted during ingestion are marked NOT FOUND, which is why the dual NLI+KG approach is necessary.

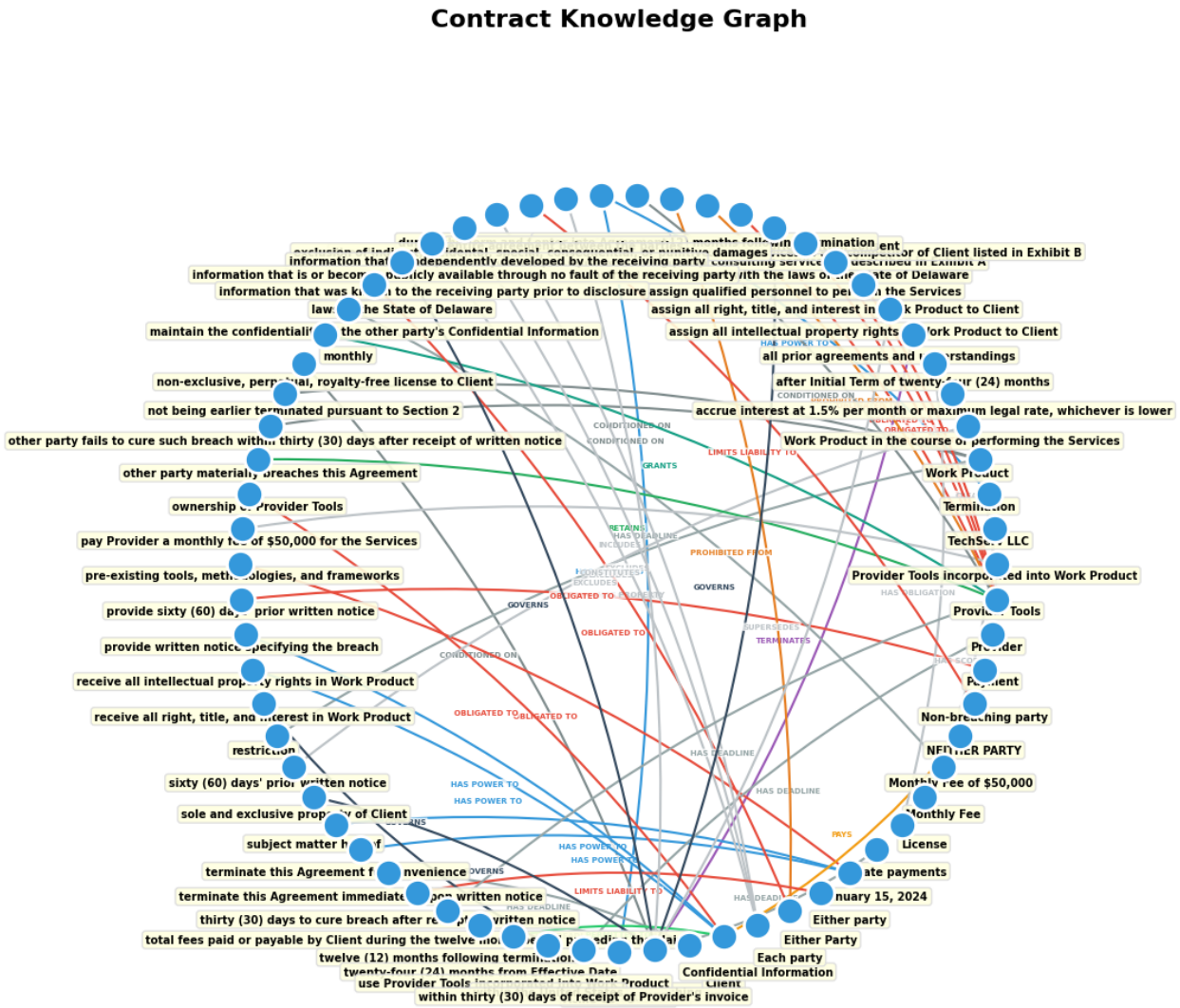


Figure 6.4: Knowledge Graph visualization extracted from a sample contract. Nodes represent entities (parties, terms, obligations) and directed edges represent Hohfeldian predicates (OBLIGATED_TO, PAYS, HAS_POWER_TO, etc.) with color-coded categories: red for obligations, blue for powers, green for permissions, and yellow for transactional relations.

6.2 Evaluation

We evaluated on 225 question-answer pairs derived from CUAD legal contracts. Each example pairs a contract passage with a question targeting a specific clause. We evaluate along three axes: answer quality, structural compliance, and verification quality. We conduct an

ablation study over six configurations of increasing complexity:

1. **B1:** Vanilla Qwen 2.5-7B-Instruct (zero-shot, no retrieval).
2. **B2:** Base Qwen + RAG (FAISS top-3 retrieval).
3. **B3:** SFT Qwen + RAG (LoRA fine-tuned on 2,700 tool-calling traces).
4. **B4:** B3 + NLI filtering (claims with entailment < 0.5 discarded).
5. **B5:** B3 + KG cross-check (claims not matching any KG triple discarded).
6. **B6:** Full RLC pipeline (B3 + NLI + KG verification).

The result can be shown in table 6.7.

Table 6.7: Ablation results on the CUAD-derived evaluation set (225 examples).

Metric	**B1**	**B2**	**B3**	**B4**	**B5**	**B6**
Answer F1	0.34	0.42	0.43	**0.45**	0.44	**0.45**
Semantic F1	0.37	0.43	0.46	0.46	0.45	**0.47**
Citation F1	0.08	0.25	**0.52**	**0.52**	**0.52**	**0.52**
Tool Format	0.00	0.00	**0.82**	**0.82**	**0.82**	**0.82**
Completeness	0.00	0.00	**0.78**	**0.78**	**0.78**	**0.78**
Grounding	0.32	0.40	0.48	0.62	0.52	**0.65**
Halluc. Rate (↓)	0.35	0.28	0.25	0.18	0.21	**0.15**
KG Consistency	–	–	–	–	0.30	**0.35**
Retrieval Prec.	–	0.55	0.55	0.55	0.55	0.55

6.3 Discussion

The results demonstrate that each component of the RLC pipeline contributes meaningfully, with SFT providing the largest absolute gains and neuro-symbolic verification providing the most targeted improvements to factual reliability. The 57% relative reduction in hallucination rate from B1 (0.35) to B6 (0.15) is particularly notable for legal applications where factual accuracy is paramount. We observe that retrieval precision (0.55) remains constant across all RAG-enabled configurations, suggesting that answer quality gains stem from improved utilization of retrieved context rather than better retrieval. A limitation of our evaluation is the moderate size of the benchmark (225 examples) and the reliance on a single contract corpus (CUAD); future work should validate on broader le- gal corpora and

adversarial settings. Additionally, the KG Consistency scores (0.30–0.35) indicate headroom for improving the triple extraction pipeline, which currently operates on automatically parsed clause structures.

7. Conclusions

This project delivers a practical and reliable pipeline for legal contract analysis. It uses Qwen 7B to generate answers, DeBERTa-v3 to verify whether claims are actually supported by the contract, and Sentence Transformers for semantic retrieval of relevant clauses. On the verification side, it adds a knowledge graph with structured triples and rule-based checks, which helps catch structural mistakes like wrong entities or amounts. The answer model results in strong structured behavior (Tool Format Score of 0.82). A major strength of the system is its dual verification design: NLI helps detect semantic hallucinations, while KG cross-checking provides structural validation. The model also follows a clear four-step reasoning flow (cite $\rightarrow$ verify $\rightarrow$ extract $\rightarrow$ summarize), making outputs easier to trace and audit, with use of core tools in evaluation. Finally, confidence gating adds a safety layer by warning users when answers are uncertain; overall, the system reached a 65% grounding rate ,with a decrease in hallucination to 15%.

7.1 Limitations

1. The NLI model's conservative scoring (average 0.396) leads to low confidence even for factually correct answers, suggesting the entailment threshold may need calibration for legal text.

2. Clause extraction depends on an external API for the demo; a self-contained module would improve portability.

3. The KG predicate schema is fixed and may not capture all relationships in specialized contract types.

4. Single-contract scope without cross-contract analysis support.

5. Computational requirements (GPU with 16+ GB VRAM) and generation latency ($\sim$27s) limit edge deployment.

6. English-only language support.

7.2 Future Work

1. **NLI Threshold Calibration**: Fine-tune the verification thresholds using a validation set to reduce false negatives (correct claims marked as WEAK).

2. **Semantic KG Matching**: Replace keyword-based matching with embedding-based triple retrieval for improved symbolic context and cross-check coverage.

3. **Multi-contract Analysis**: Extend to comparative analysis across multiple contracts.

4. **Smaller Model Distillation**: Distill the fine-tuned Qwen 7B into a 1–3B parameter model for faster inference and edge deployment.

5. **Active Learning**: Use user feedback on verification results to continuously improve the NLI model.

6. **Multi-language Support**: Extend to contracts in multiple languages using multilingual models.

7. **Temporal Reasoning**: Add explicit handling of date-dependent clauses and deadline calculations.

8. **Larger Evaluation Set**: Evaluate on larger and well annotated dataset for a more more reliable system quality validation

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS)*, pages 9459–9474, 2020.

[2] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781. Association for Computational Linguistics, 2020.

[3] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992. Association for Computational Linguistics, 2019.

[4] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[5] Wesley Newcomb Hohfeld. Some fundamental legal conceptions as applied in judicial reasoning. *Yale Law Journal*, 23(1):16–59, 1913.

[6] Bodhisattwa Prasad Gutierrez, Nikolas McNeal, Derek Washington, Yifan Chen, Xinya Li, Zhengping Jiang, and Heng Ji. From text to knowledge graph: Large language models for schema-rich information extraction. *arXiv preprint arXiv:2404.02072*, 2024.

[7] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599, 2024.

[8] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[9] Andrew Blair-Stanek, Nils Holzenberger, and Benjamin Van Durme. Can gpt-3 perform statutory reasoning? In *Proceedings of the 19th International Conference on Artificial Intelligence and Law (ICAIL)*, pages 22–31. ACM, 2023.

[10] Jaromir Savelka, Kevin D. Ashley, Morgan A. Gray, Hannes Westermann, and Huihui Xu. Explaining legal concepts with augmented large language models (GPT-4). In *Proceedings of the 36th International Conference on Legal Knowledge and Information Systems (JURIX)*, pages 209–218. IOS Press, 2023.

[11] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *Proceedings of the 9th International Conference on Learning Representations (ICLR)*, 2021.

[12] Or Honovich, Roee Aharoni, Jonathan Herzig, Hagai Taitelbaum, Doron Kuber, Vered Shaar, Thomas Schuster, Doudou Hou, Siddhartha Gupta, Idan Szpektor, et al. TRUE: Re-evaluating factual consistency evaluation. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 3905–3920. Association for Computational Linguistics, 2022.

[13] Philippe Laban, Tobias Schnabel, Paul N. Bennett, and Marti A. Hearst. SummaC: Re-visiting NLI-based models for inconsistency detection in summarization. *Transactions of the Association for Computational Linguistics*, 10:163–177, 2022.

[14] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. FEVER: A large-scale dataset for fact extraction and VERification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 809–819. Association for Computational Linguistics, 2018.

[15] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.

[16] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[17] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In

Proceedings of the 10th International Conference on Learning Representations (ICLR), 2022.

[18] Ilias Chalkidis, Manos Fergadiotis, Prodromos Malakasiotis, Nikolaos Aletras, and Ion Androutsopoulos. LEGAL-BERT: The muppets straight out of law school. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2898–2904. Association for Computational Linguistics, 2020.

[19] Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. CUAD: An expert-annotated NLP dataset for legal contract review. In *Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.

[20] Michael James Bommarito II and Daniel Martin Katz. Gpt takes the bar exam. *arXiv preprint arXiv:2212.14402*, 2022.

[21] Nirmalie Wiratunga, Ramitha Abeyratne, Lasal Jayawardena, Kyle Martin, Stewart Massie, Ikechukwu Nkisi-Orji, Ruvan Weerasinghe, Anne Liret, and Bruno Fleisch. RAG for legal question answering. In *Proceedings of the SIGIR 2024 Workshop on Legal Information Retrieval*, 2024.

[22] Antoine Louis, Gijs van Dijck, and Gerasimos Spanakis. Legal retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*. Association for Computational Linguistics, 2024.

[23] Ho-Pun Lam, Mustafa Hashmi, and Bill Scofield. Building a legal knowledge graph for smart compliance checking. In *Proceedings of the 18th International Conference on Artificial Intelligence and Law (ICAIL)*, pages 81–90. ACM, 2020.

[24] Mauro Dragoni, Serena Villata, Williams Rizber, and Guido Governatori. Combining nlp approaches for rule extraction from legal documents. In *1st Workshop on MIning and REasoning with Legal texts (MIREL)*, 2016.

[25] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9004–9017. Association for Computational Linguistics, 2023.

[26] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained

atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 12076–12100. Association for Computational Linguistics, 2023.

[27] Hannah Sansford, Nicholas Richardson, Hermina Petric Maretic, and Juba Nait Saada. GraphEval: A knowledge-graph based LLM hallucination evaluation framework. *arXiv preprint arXiv:2407.10793*, 2024.

Appendix A. Source code

A.1 Clause Extraction with Claude API

Claude API segments the raw contract into structured clauses, preserving exact original text.

```
import anthropic

client = anthropic.Anthropic(api_key=ANTHROPIC_API_KEY)

CLAUSE_EXTRACTION_PROMPT = """You are a legal document
segmentation engine. Split this contract into its individual
clauses/sections.

For each clause, extract:
- clause_number: the numbering as it appears
- title: the clause heading/title
- text: the COMPLETE text of the clause

Return ONLY a JSON array:
[{"clause_number": "1", "title": "Definitions",
  "text": "full clause text here..."}]

Rules:
- Preserve the EXACT original text. Do not paraphrase.
- Include ALL clauses."""

response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=4096,
    temperature=0.0,
    system=CLAUSE_EXTRACTION_PROMPT,
    messages=[{"role": "user", "content": DEMO_CONTRACT}],
)

raw_clauses = response.content[0].text
match = re.search(r"\[.*\]", raw_clauses, re.DOTALL)
clauses_data = json.loads(match.group(0)) if match else []
```

Listing A.1: Clause Extraction using Claude API

A.2 Triple Extraction and Knowledge Graph Construction

For each clause, Claude extracts structured triples using the Hohfeldian predicate schema.

```
TRIPLE_EXTRACTION_PROMPT = """Extract structured facts (triples)
```

```
from this legal clause.

For each fact, provide:
- subject: the entity (party name, "Agreement", etc.)
- predicate: the relationship (UPPER_SNAKE_CASE:
  OBLIGATED_TO, PERMITTED_TO, PROHIBITED_FROM,
  HAS_POWER_TO, PAYS, DELIVERS, TERMINATES,
  GOVERNS, HAS_DEADLINE, CONDITIONED_ON,
  LIMITS_LIABILITY_TO)
- object: what the predicate applies to

Return ONLY valid JSON:
{"triples": [{"subject": "Buyer",
  "predicate": "OBLIGATED_TO",
  "object": "pay within 30 days"}]}"""

all_triples = []
for clause in clauses:
    resp = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=1024,
        temperature=0.0,
        system=TRIPLE_EXTRACTION_PROMPT,
        messages=[{"role": "user",
            "content": f"Clause {clause.clause_id}:\n"
                       f"{clause.text}"}],
    )
    raw = resp.content[0].text
    match = re.search(r"\{.*\}", raw, re.DOTALL)
    if match:
        data = json.loads(match.group(0))
        for item in data.get("triples", []):
            triple = Triple(
                subject=item["subject"],
                predicate=item["predicate"],
                object=item["object"],
                clause_id=clause.clause_id,
            )
            all_triples.append(triple)
```

Listing A.2: Triple Extraction for Knowledge Graph

A.3 FAISS Clause Retrieval

Clauses are encoded into 384-dimensional vectors using SentenceTransformers and indexed with FAISS for cosine similarity search.

```
from sentence_transformers import SentenceTransformer
import faiss

embed_model = SentenceTransformer("all-MiniLM-L6-v2")

# Encode all clauses
```

```
clause_texts = [f"{c.title} {c.text}" if c.title
                else c.text for c in clauses]
clause_embeddings = embed_model.encode(
    clause_texts, normalize_embeddings=True
)

# Build FAISS index (cosine similarity via inner product)
dim = clause_embeddings.shape[1]
faiss_index = faiss.IndexFlatIP(dim)
faiss_index.add(clause_embeddings.astype(np.float32))

def retrieve_clauses(question, top_k=5, threshold=0.3):
    """Retrieve top-k relevant clauses for a question."""
    query_emb = embed_model.encode(
        [question], normalize_embeddings=True
    ).astype(np.float32)
    scores, indices = faiss_index.search(
        query_emb, min(top_k, len(clauses))
    )
    results = []
    for score, idx in zip(scores[0], indices[0]):
        if idx >= 0 and score >= threshold:
            results.append((clauses[idx], float(score)))
    return results
```

Listing A.3: FAISS Index Construction and Retrieval

A.4 KG Symbolic Context Assembly

Before answer generation, relevant KG triples are assembled into structured context by keyword matching.

```
def get_symbolic_context(question, triples):
    """Build symbolic context from KG triples relevant
    to the question."""
    words = [w.lower() for w in question.split()
             if len(w) > 3]
    related = []

    for triple in triples:
        triple_text = (f"{triple.subject} "
                       f"{triple.predicate} "
                       f"{triple.object}").lower()
        for word in words:
            if word in triple_text:
                related.append(triple)
                break

    if not related:
        return "(No directly relevant KG facts found)"

    seen = set()
    lines = ["Knowledge Graph Facts:"]
```

```
    for t in related:
        key = (t.subject, t.predicate, t.object)
        if key not in seen:
            seen.add(key)
            lines.append(
                f"  ({t.subject})-[{t.predicate}]"
                f"->({t.object}) [from {t.clause_id}]"
            )
    return "\n".join(lines)
```

Listing A.4: Symbolic Context Assembly from Knowledge Graph

A.5 Answer Generation with Fine-Tuned Qwen 7B

The fine-tuned Qwen 2.5 7B model with SFT LoRA adapter generates structured tool-call responses.

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel

# Load base model + SFT LoRA adapter
tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL)
model = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL, device_map="auto", torch_dtype=torch.bfloat16
)
model = PeftModel.from_pretrained(model, SFT_ADAPTER_PATH)
model.eval()

SYSTEM_PROMPT = """You are a legal contract analysis agent
with tools for verification.

You have these tools:
1. cite_clause(quote, section)
2. verify_claim(claim, evidence, claim_type)
3. extract_fact(fact_name, fact_value, confidence, evidence)
4. summarize_analysis(answer, confidence, key_findings)

Always: cite_clause first, then verify_claim for each claim,
then summarize_analysis last."""

def generate_answer(question, retrieved_clauses,
                    symbolic_context="", max_new_tokens=512):
    """Generate a structured answer using fine-tuned Qwen."""
    clause_text = "\n\n".join(
        f"--- Clause {c.clause_id} ({c.section}) ---\n"
        f"{c.text}" for c, _ in retrieved_clauses
    )
    user_content = f"CONTRACT CLAUSES:\n{clause_text}"
    if symbolic_context:
        user_content += (f"\n\nKNOWLEDGE GRAPH CONTEXT:\n"
                         f"{symbolic_context}")
    user_content += f"\n\nQUESTION: {question}"

```

```
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": user_content},
    ]
    text = tokenizer.apply_chat_template(
        messages, tokenize=False, add_generation_prompt=True
    )
    inputs = tokenizer(text, return_tensors="pt",
                       truncation=True, max_length=2048
                       ).to(model.device)

    with torch.no_grad():
        output = model.generate(
            **inputs, max_new_tokens=max_new_tokens,
            temperature=0.1, do_sample=True, top_p=0.9,
            pad_token_id=tokenizer.pad_token_id,
        )
    response = tokenizer.decode(
        output[0][inputs["input_ids"].shape[1]:],
        skip_special_tokens=True
    )
    return response
```

Listing A.5: Model Loading and Answer Generation

A.6 NLI Verification

The fine-tuned DeBERTa-v3 NLI model scores each claim against contract text for entailment.

```
from transformers import (AutoTokenizer as AT,
    AutoModelForSequenceClassification)

nli_tokenizer = AT.from_pretrained(NLI_MODEL_PATH)
nli_model = AutoModelForSequenceClassification\
    .from_pretrained(NLI_MODEL_PATH)
nli_model.to(DEVICE).eval()

NLI_LABELS = ["contradiction", "entailment", "neutral"]

def nli_score(premise, hypothesis):
    """Score (premise, hypothesis) -> entailment prob."""
    inputs = nli_tokenizer(
        premise, hypothesis,
        truncation=True, max_length=512,
        return_tensors="pt"
    ).to(DEVICE)

    with torch.no_grad():
        probs = torch.softmax(
            nli_model(**inputs).logits, dim=-1
        )[0]

```

```
    p_con = float(probs[0])
    p_ent = float(probs[1])
    p_neu = float(probs[2])

    return {
        "entailment": round(p_ent, 4),
        "contradiction": round(p_con, 4),
        "neutral": round(p_neu, 4),
        "grounding_score": round(p_ent, 4),
    }
```

Listing A.6: NLI Cross-Encoder Verification

A.7 KG Cross-Check

The KG cross-check verifies claims against structured facts in the knowledge graph.

```
def kg_cross_check(claim, triples):
    """Check if a claim is consistent with KG facts."""
    claim_lower = claim.lower()

    for triple in triples:
        subj_in = triple.subject.lower() in claim_lower
        obj_in = triple.object.lower() in claim_lower
        pred_words = triple.predicate.lower()\
            .replace("_", " ")

        if subj_in and obj_in:
            return True, (f"({triple.subject})"
                f"-[{triple.predicate}]"
                f"->({triple.object})")
        if (subj_in or obj_in) and \
                pred_words in claim_lower:
            return True, (f"({triple.subject})"
                f"-[{triple.predicate}]"
                f"->({triple.object})")

    return False, None
```

Listing A.7: KG Cross-Check for Structural Consistency

A.8 Multi-Signal Verification and Confidence Gating

The unified verification pipeline combines NLI and KG signals with confidence gating.

```
def verify_claims(claims, clause_texts, triples,
                  contract_id):
    """Full multi-signal verification: NLI + KG."""
    full_premise = " ".join(clause_texts)
    verified = []

    for claim in claims:
        # Signal 1: NLI entailment
```

```
        nli_result = nli_score(
            premise=full_premise[:2000],
            hypothesis=claim
        )
        nli_s = nli_result["grounding_score"]

        # Signal 2: KG cross-check
        kg_ok, kg_evidence = kg_cross_check(
            claim, triples
        )

        # Combine signals (decision matrix)
        if kg_ok and nli_s > 0.5:
            method = VerificationMethod.NLI_KG
            is_verified = True
        elif nli_s > 0.7:
            method = VerificationMethod.NLI
            is_verified = True
        elif kg_ok:
            method = VerificationMethod.KG
            is_verified = True
        elif nli_s > 0.4:
            method = VerificationMethod.WEAK
            is_verified = False
        else:
            method = VerificationMethod.UNVERIFIED
            is_verified = False

        verified.append(VerifiedClaim(
            claim=claim, verified=is_verified,
            method=method, nli_score=nli_s,
            kg_verified=kg_ok,
        ))
    return verified

def compute_confidence(verified_claims):
    """Compute overall confidence from verification."""
    method_weights = {
        VerificationMethod.NLI_KG: 1.0,
        VerificationMethod.NLI: 0.9,
        VerificationMethod.KG: 0.8,
        VerificationMethod.WEAK: 0.6,
        VerificationMethod.UNVERIFIED: 0.4,
    }
    total_weight, weighted_sum = 0.0, 0.0
    for vc in verified_claims:
        w = method_weights.get(vc.method, 0.5)
        weighted_sum += vc.nli_score * w
        total_weight += w
    return weighted_sum / max(total_weight, 1e-6)
```

Listing A.8: Multi-Signal Verification Pipeline

A.9 Full End-to-End Pipeline

The complete pipeline orchestrates all seven phases sequentially.

```python
def run_full_pipeline(question, clauses_list,
                      triples_list, top_k=5):
    """Run the complete RLC pipeline end-to-end."""
    # Step 1: Retrieve relevant clauses
    retrieved = retrieve_clauses(question, top_k=top_k)
    clause_texts = [c.text for c, _ in retrieved]

    # Step 2: Get KG symbolic context
    sym_ctx = get_symbolic_context(
        question, triples_list
    )

    # Step 3: Generate answer with fine-tuned Qwen 7B
    response, gen_time = generate_answer(
        question, retrieved, sym_ctx
    )
    tool_steps = parse_tool_calls(response)
    answer = extract_final_answer(response, tool_steps)

    # Step 4: Extract claims for verification
    claims = []
    for step in tool_steps:
        if step.tool_name == "verify_claim":
            c = step.tool_input.get("claim", "")
            if c: claims.append(c)
        elif step.tool_name == "extract_fact":
            r = step.tool_input.get("reasoning", "")
            if r: claims.append(r)

    # Step 5: NLI + KG verification
    verified = verify_claims(
        claims, clause_texts, triples_list, CONTRACT_ID
    )
    confidence = compute_confidence(verified)

    # Step 6: Confidence gating
    n_grounded = sum(1 for vc in verified if vc.verified)
    if confidence < 0.5 and verified:
        answer += (
            f"\n\nWARNING: Low confidence "
            f"({confidence:.0%}): "
            f"{len(verified) - n_grounded} of "
            f"{len(verified)} claims could not "
            f"be verified."
        )

    return {
        "question": question,
        "answer": answer,
        "verified_claims": verified,
```

```
        "confidence": confidence,
        "n_grounded": n_grounded,
        "n_total": len(verified),
    }
```

Listing A.9: Full RLC Pipeline — End-to-End Execution

Appendix B. Dataset Statistics

Complete Dataset Statistics

Dataset	**Purpose**	**Train**	**Eval**
SFT Training			
curated_20	High-quality tool-call examples	300	75
psl_train/eval	Extended SFT data	450	112
train_unsloth	Unsloth-formatted SFT data	300	75
GRPO Training			
grpo_train	GRPO prompts (category-balanced)	300	—
grpo_eval	GRPO evaluation	—	225
NLI Training			
cuad_nli_train	Legal NLI pairs (gold + weak)	8,655	—
cuad_nli_st_train	Filtered high-quality pairs	2,190	161
Earlier Iterations (not used in final model)			
final_merged	Combined reasoning data	1,200	300
cuad_reasoning	CUAD-based reasoning chains	800	200
claude_code_generated	Synthetic data from Claude	400	100